MinIO Object Storage Integration

<details>
<summary>Relevant source files</summary>

- `Backend/STUDER-MEDIA/Dockerfile`
- `Backend/STUDER-MEDIA/main.py`
- `Backend/STUDER-MEDIA/placeholder.txt`
- `Backend/STUDER/Dockerfile`
- `Backend/STUDER/pom.xml`
- `Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java`
- `Backend/STUDER/src/main/java/facu/studer/config/AppConfig.java`
- `Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java`
- `Backend/STUDER/src/main/java/facu/studer/config/WebClientConfig.java`
- `Docker/docker-compose.yml`
</details>

The STUDER platform utilizes **MinIO** as its object storage solution, providing an S3-compatible interface for managing media assets. This integration is primarily handled by the `STUDER-MEDIA` microservice, which manages bucket lifecycle, access policies, and image persistence.

Configuration & Environment

The connection to MinIO is established using environment variables defined in the infrastructure layer and consumed by the Python FastAPI service.

Variable	Description	Source
`MINIO_ENDPOINT`	The network address of the MinIO service (e.g., `minio:9000`).	`Docker/docker-compose.yml` 45
`MINIO_ROOT_USER`	Access key for authentication.	`Docker/docker-compose.yml` 46
`MINIO_ROOT_PASSWORD`	Secret key for authentication.	`Docker/docker-compose.yml` 47

Variable	Description	Source
`BUCKET_NAME`	The default bucket for all media (defaults to `images`).	`Docker/docker-compose.yml` 48

S3 Client Initialization

The `STUDER-MEDIA` service uses the `boto3` library to interface with MinIO. The client is initialized lazily via the `get_s3_client` function, which constructs the endpoint URL using the `http://` prefix and the configured endpoint `Backend/STUDER-MEDIA/main.py` 30-40

Sources:

- `Docker/docker-compose.yml` 45-48
- `Backend/STUDER-MEDIA/main.py` 19-26
- `Backend/STUDER-MEDIA/main.py` 30-40

Bucket Lifecycle & Security

To ensure the system is ready upon deployment, the media service implements automated bucket initialization with a retry mechanism and public access configuration.

Initialization Flow

During the FastAPI `startup` event, the service executes `create_bucket_if_not_exists` `Backend/STUDER-MEDIA/main.py` 93-95

1. **Retry Logic**: The service attempts to connect to MinIO up to 5 times with a 5-second delay between attempts to account for container startup lag `Backend/STUDER-MEDIA/main.py` 62-63 `Backend/STUDER-MEDIA/main.py` 88-89
2. **Existence Check**: It uses `s3_client.head_bucket` to verify if the bucket exists `Backend/STUDER-MEDIA/main.py` 65
3. **Creation**: If a `404` error is returned, it creates the bucket using `s3_client.create_bucket` `Backend/STUDER-MEDIA/main.py` 70-72

4. **Policy Enforcement**: Once created or verified, it applies a public-read policy
   `Backend/STUDER-MEDIA/main.py 67`

Public Read Policy

To allow the frontend and external clients to view images without signed URLs, the `set_public_read_policy` function applies a JSON policy to the bucket `Backend/STUDER-MEDIA/main.py 42-53` This policy grants `s3:GetObject` permissions to all principals (`*`) for all resources within the configured bucket `Backend/STUDER-MEDIA/main.py 43-53`

Sources:

- `Backend/STUDER-MEDIA/main.py 42-59`
- `Backend/STUDER-MEDIA/main.py 60-91`
- `Backend/STUDER-MEDIA/main.py 93-95`

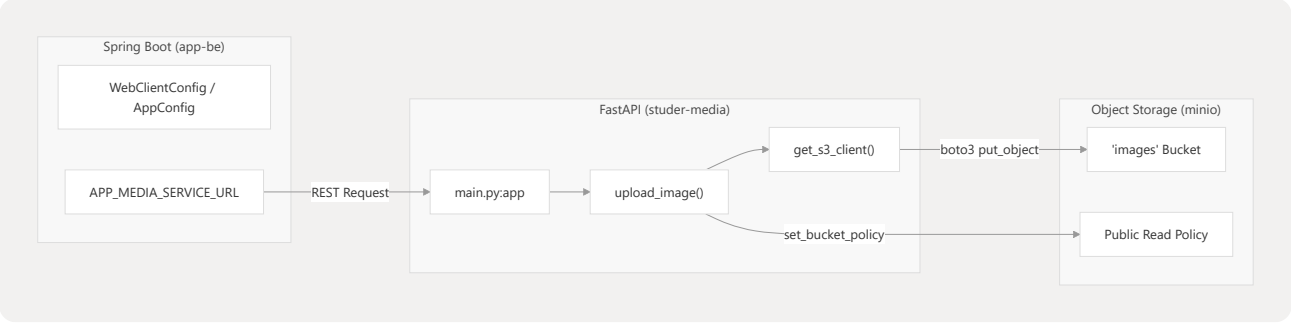
Data Flow: Backend to MinIO

The Spring Boot backend (`STUDER`) does not communicate with MinIO directly. Instead, it acts as a coordinator, delegating image processing and storage tasks to the `STUDER-MEDIA` service.

Integration Diagram

The following diagram illustrates the relationship between the Spring Boot configuration and the Media Service entities.

Entity Mapping: Backend Configuration to Media Storage



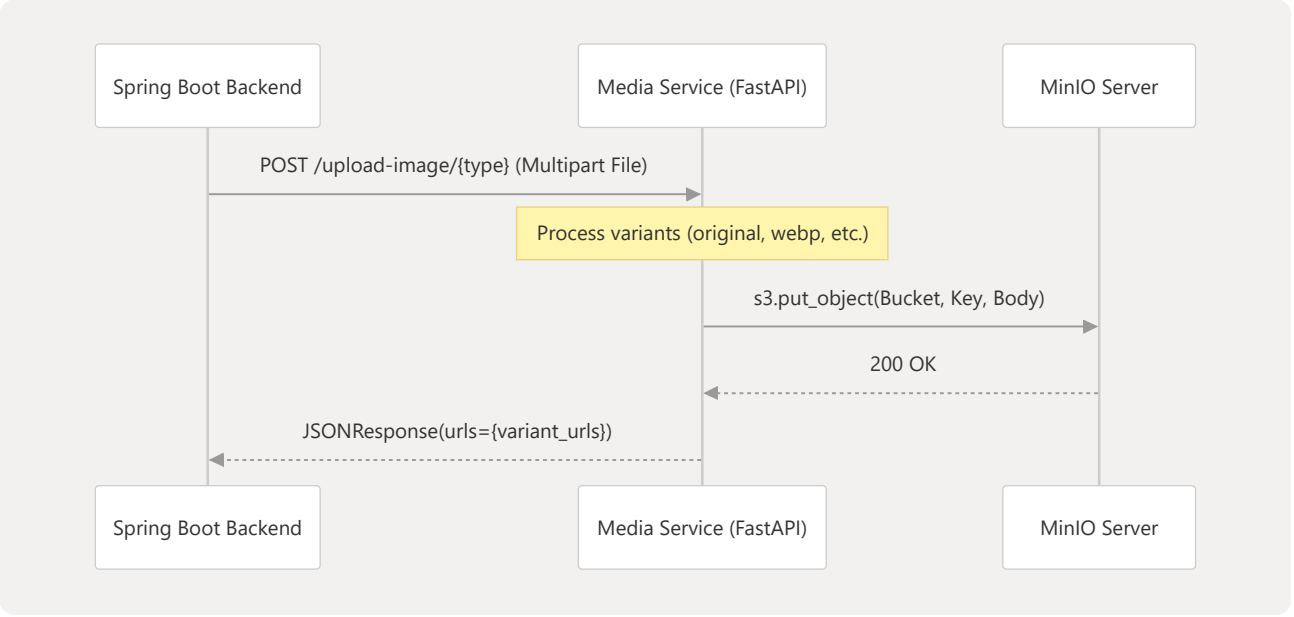
Sources:

- `Docker/docker-compose.yml` `30`
- `Backend/STUDER/src/main/java/facu/studer/config/AppConfig.java` `11-13`
- `Backend/STUDER-MEDIA/main.py` `97-102`

Service Communication

The backend is configured with the **`APP_MEDIA_SERVICE_URL`** environment variable (typically **`http://studer-media:8000`**), which it uses to target the media microservice `Docker/docker-compose.yml` `30` Requests are sent using a **`RestTemplate`** bean defined in **`AppConfig`** `Backend/STUDER/src/main/java/facu/studer/config/AppConfig.java` `10-14`

Storage Interaction Logic



Sources:

- `Backend/STUDER-MEDIA/main.py` `113-144`
- `Backend/STUDER-MEDIA/main.py` `169`
- `Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java` `42-48`

URL Generation

When images are successfully uploaded, the media service returns a map of URLs. These URLs are constructed using the internal MinIO port for processing, but they are typically served via the reverse proxy in production. In the current implementation, the URLs point to **`http://localhost:9000/{BUCKET_NAME}/...`** `Backend/STUDER-MEDIA/main.py 114` The backend stores these strings in fields like **`profilePictureOriginalUrl`** and **`profilePictureThumbnailUrl`** `Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java 42-48`

Sources:

- `Backend/STUDER-MEDIA/main.py 114-144`
- `Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java 42-48`